Theoretical Computer Science

Regular Language & Regular Grammar

Vietnamese-German University

Lecturers: Ngoc Tran, Ph.D.

Viet To, Ph.D.

Contents

- Regular Expression
- Connection between Regular Expressions and Regular Languages
- Regular Grammars
- Pumping Lemma

Regular Expression

- *A language is regular if there exists a finite accepter for it.* Therefore, *every regular language can be described by some dfa or some nfa*.
- In many instances, we need more concise ways of describing regular languages. The other ways of representing regular languages are named *regular expression* and *regular grammar*.

Regular Expression

- Is a combination of strings of symbols from some alphabet Σ, parentheses ( ), and the operators +, ., and *.
  - We use $\cdot$ for concatenation, and * for star-closure, and the + for union.
- The simplest case is the language $\{a\}$, denoted by the regular expression a.
- Slightly more complicated is the language $\{a, b, c\}$, for which, we have the regular expression $a+b+c$.
- The expression $(a+(b\cdot c))^*$ stands for the star-closure of $\{a\}\cup\{bc\}$, that is, the language $\{\lambda, a, bc, aa, abc, bca, bcbc, aaa, aabc, \ldots\}$.

Regular Expression

- Definition 3.1: Let Σ be a given alphabet. Then
  1. $\emptyset$, λ and $a \in \Sigma$ are all regular expressions. These are called primitive regular expressions.
  2. If r_1 and r_2 are regular expressions, so are $r_1 + r_2$, $r_1.r_2$, r_1^*, and (r_1).
  3. A string is a regular expression if and only if it can be derived from the primitive regular expressions by a finite number of applications of the rules in the step 2.

Regular Expression

Example

- For $\Sigma = \{a, b, c\}$, the string $(a+b+c)^*.(c+\emptyset)$ is a regular expression since it is constructed by application of the above rules.
- If we take $r_1 = c$ and $r_2 = \emptyset$, we find that $c + \emptyset$ and $(c + \emptyset)$ are also regular expressions. Repeating this, we eventually generate the whole string.
- On the other hand, $(a + b +)$ is not a regular expression, since there is no way it can be constructed from the primitive regular expressions.

Regular Expression

Languages Associated with Regular Expressions

Definition 3.2: The language $L(r)$ denoted by any regular expression r is defined by the following rules.

1. $\emptyset$ is a regular expression denoting the empty set,
2. λ is a regular expression denoting $\{\lambda\}$.
3. For every $a \in \Sigma$, a is a regular expression denoting $\{a\}$.

Regular Expression

Languages Associated with Regular Expressions

If r_1 and r_2 are regular expressions, then

1. $L(r_1 + r_2) = L(r_1) \cup L(r_2)$
2. $L(r_1 \cdot r_2) = L(r_1) \,.\, L(r_2)$
3. $L((r_1)) = L(r_1)$
4. $L(r_1^*) = (L(r_1))^*$

Regular Expression

- Priority on the operators from left to right: *, ., +
- The symbol for concatenation may be omitted, so we can write $r_1 r_2$ for $r_1.r_2$.
- Example: Exhibit the language $L(a^* \cdot (a + b))$ in set notation.

$$
\begin{aligned}
L(a^* \,.\, (a + b)) &= L(a^*)\, L(a + b) \\
&= (L(a))^* \,(L(a) \cup L(b)) \\
&= \{\lambda, a, aa, aaa, \ldots\}\{a, b\} \\
&= \{a, aa, aaa, \ldots, b, ab, aab, \ldots\}
\end{aligned}
$$

Regular Expression

Example:

For $\Sigma = \{a, b\}$, the expression

$$r_0 = (a+b)^*(a+bb)$$

is regular. It denotes the language

$$L(r_0) = \{a, bb, aa, abb, ba, bbb, \ldots\}$$

Regular Expression

Find the languages denoted by the following regular expressions:

$r_1 = (aa)^*(bb)^*b$

$r_2 = (ab^*a + b)^*$

Results:

$L(r_1) = \{a^{2n}b^{2m+1}: n \geq 0, m \geq 0\}$

$L(r_2) = \{w \in \{a, b\}^*: n_a(w) \text{ is even}\}$

Regular Expression

- Find a regular expression for the language

$L = \{w \in \{0,1\}^* : w$ has no pair of consecutive zeros$\}$

- Answer:

$r = (1^*011^*)^* (0 + \lambda) + 1^* (0 + \lambda)$

$r = (1 + 01)^* (0 + \lambda)$

- Equivalence of regular expressions: the two regular expressions are equivalent if they denote the same language.

Regular Expression

Find a regular expression for the language

L_1 = {a set of real numbers in C}

$L_2 = \{w \in \{0, 1\}^*: n_0(w) = n_1(w)\}$

Results:

$r_1 = (\text{‘+’} + \text{‘-’} + \lambda)(0 + 1 + \ldots + 9)^*(\text{‘.’}\ (0 + 1 + \ldots + 9)^* + \lambda)$
$(\text{‘E’}\ (\text{‘+’} + \text{‘-’} + \lambda)(0 + 1 + \ldots + 9)^* + \lambda)$

- There does not exist a regular r_2 expression for L_2

Regular Expression

Note:

- $(r^*)^* = r^*$
- $(r_1{}^* + r_2)^* = (r_1 + r_2)^*$
- $(r_1 r_2{}^* + r_2)^* = (r_1 + r_2)^*$

Regular Expression

Exercises

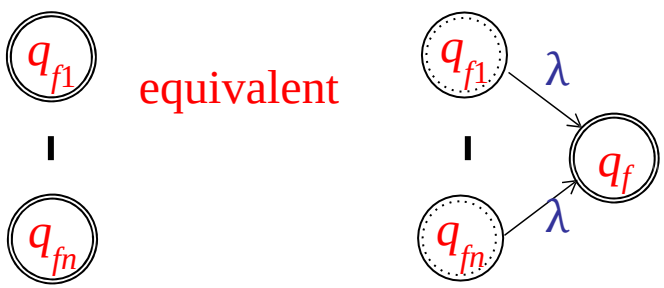
1. Find all strings in $L((a + b)b(a + ab)^*)$ of length less than four.
2. For $\Sigma = \{0, 1\}$, given $L(r) = \{w \in \Sigma^*$: w has at least one pair of consecutive zeros$\}$.
   Does the expression
   $$r = ((0 + 1)\ (0 + 1)^*)^*\ 00\ (0 + 1)^*$$
   denote the language $L(r)$.
3. Find a regular expression for $L(r) = \{a^n b^m: n \geq 3, m \text{ is even}\}$.
4. Find a regular expression for the set $L(r) = \{a^n b^m: (n + m) \text{ is even}\}$.

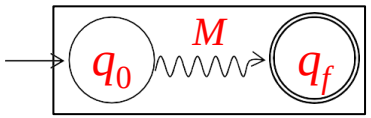
Connection between Regular Expressions and Regular Languages

Regular Expressions Denote Regular Languages

Theorem 3.1: Let r be a regular expression. Then there exists some nondeterministic finite accepter (nfa) that accepts $L(r)$. Consequently, $L(r)$ is a regular language.

- Any nfa having more than one final state has an equivalent nfa with one final state:





For every nfa, we have this chart.

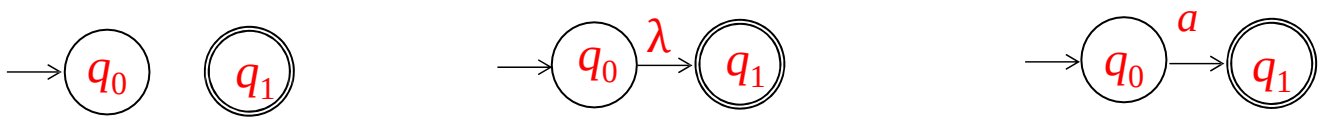
Connection between Regular Expressions and Regular Languages

To convert a regular expression to nfa, we use

Procedure: re2nfa()

- Input: Regular expression r.
- Output: nfa $M = (Q, \Sigma, \delta, q_0, F)$.

S1) Build up nfas by primitive regular expressions:

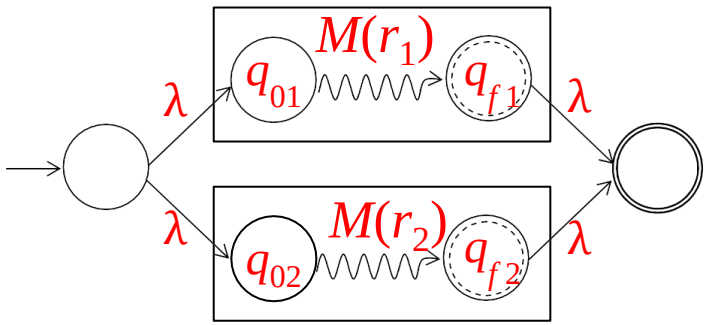


(a) nfa accepts $\emptyset$ (b) nfa accepts $\{\lambda\}$ (c) nfa accepts $\{a\}$

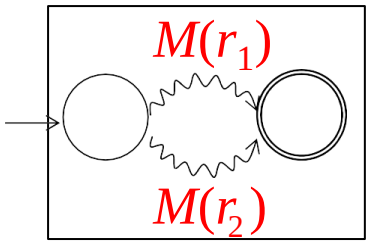
Connection between Regular Expressions and Regular Languages

S2) Build up nfas by the complex regular expressions:

- A nfa for the regular expression $r_1 + r_2$



or

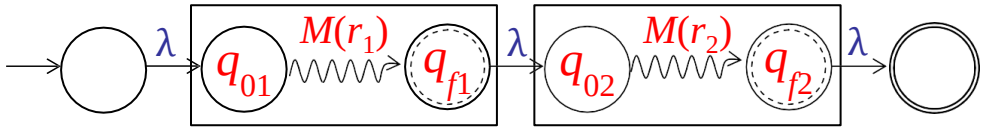


Conditions

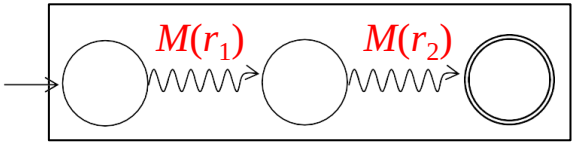
1. No edge into q_{01} and q_{02}
2. No edge out from q_{f1} and q_{f2}

Connection between Regular Expressions and Regular Languages

- A nfa for the regular expression $r_1 r_2$



or

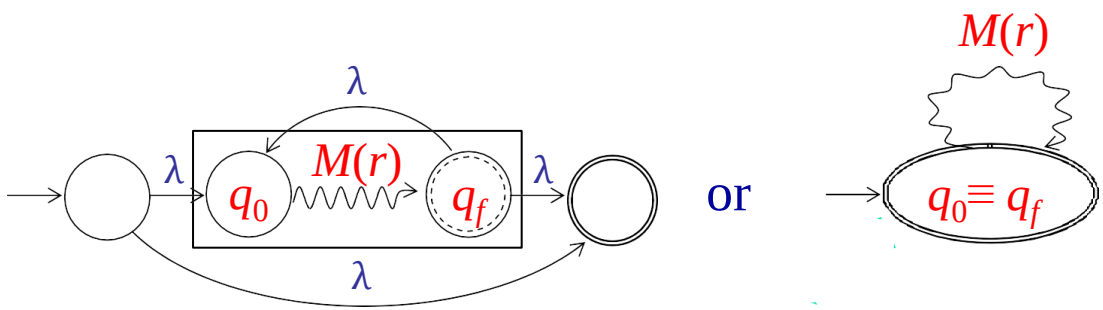


Conditions

1. No edge out from q_{f1} or q_{f2}
2. No edge in to q_{01}, q_{02}

Connection between Regular Expressions and Regular Languages

- A nfa for the regular expression r^*

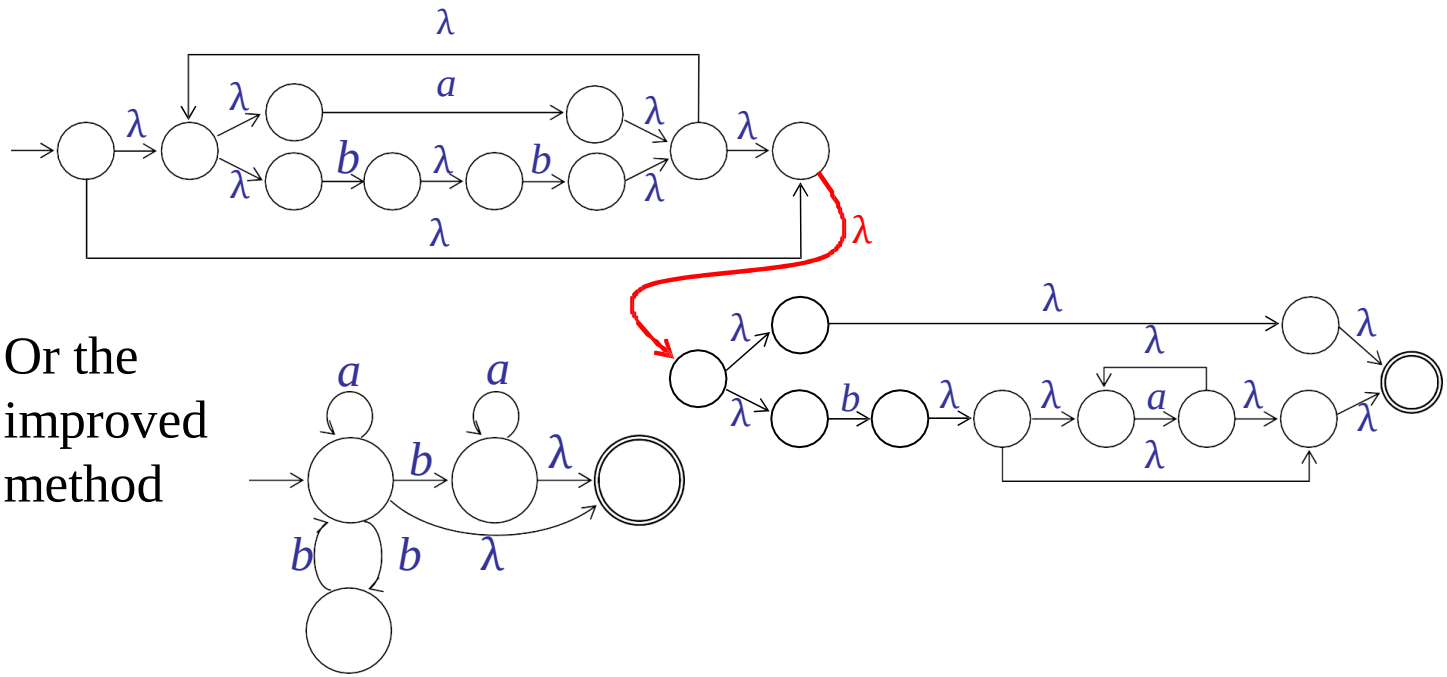


Conditions

1. No edge in to q_0
2. No edge out from q_f

Connection between Regular Expressions and Regular Languages

- A nfa for the regular expression $r = (a + bb)^*(ba^* + \lambda)$



Or the improved method

Connection between Regular Expressions and Regular Languages

Exercises

Build up nfas for the following regular expressions:

$r_1 = aa^* + aba^*b^*$

$r_2 = ab(a + ab)^* (b + aa)$

$r_3 = ab^*aa + bba^*ab$

$r_4 = a^*b(ab + b)^*a^*$

$r_5 = (ab^* + a^*b)(a + b^*a)^* b$

$r_6 = (b + a^*)(ba^* + ab)^*(b^*a + ab)$

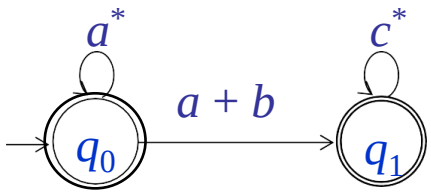
Connection between Regular Expressions and Regular Languages

Generalized Transition Graphs (GTG):

- A *generalized transition graph (GTG)* is a transition graph whose edges are labeled with regular expressions; otherwise, it is the same as the usual transition graph.
- The label of any walk from the initial state to a final state is the concatenation of several regular expressions, and hence itself a regular expression.
- The strings denoted by such regular expressions are a subset of the language accepted by the GTG, with the full language being the union of all such generated subsets.

Connection between Regular Expressions and Regular Languages

- This is an example of a GTG.
- The language accepted by it is $L(a^*+a^*(a+b)c^*)$.



- The graph of any nfa can be considered a GTG if the edge labels are interpreted properly.
  - An edge labeled with a single symbol a is interpreted as an edge labeled with the expression a.
  - An edge labeled with multiple symbols a, b,… is interpreted as an edge labeled with the expression $a + b + \ldots$.

Connection between Regular Expressions and Regular Languages

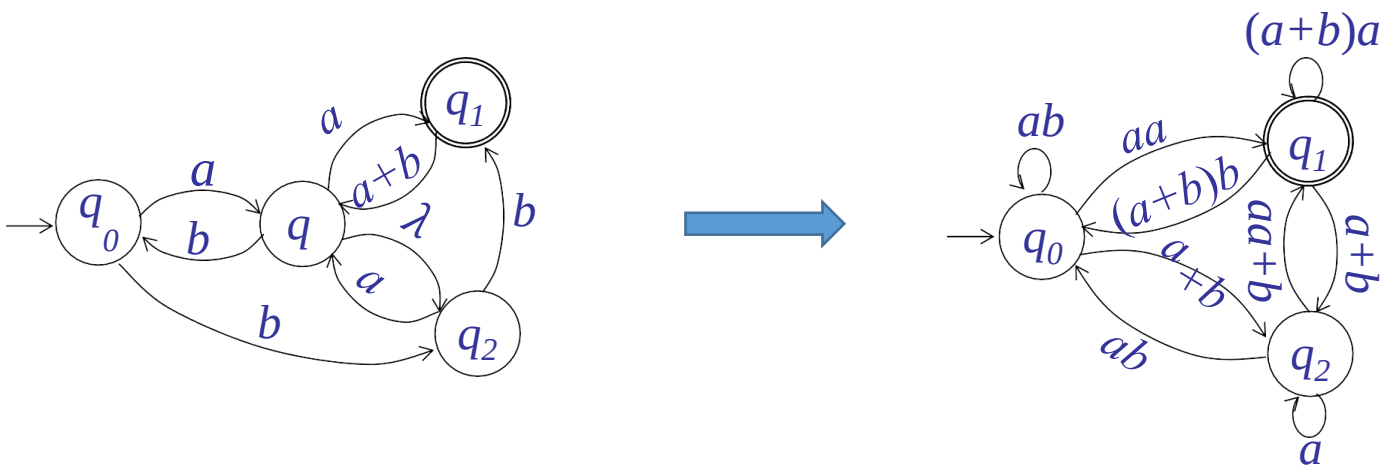
- For every regular language, there exists a GTG that accepts it. Conversely, every language accepted by a GTG is regular.
  - Since the label of every walk in a GTG is a regular expression, this appears to be an immediate consequence of Theorem 3.1

Connection between Regular Expressions and Regular Languages

- To find a regular expression for a GTG, we need to remove the intermediate states to the simplest equivalent GTG.
- Immediate states: are neither initial states nor final states.

Connection between Regular Expressions and Regular Languages

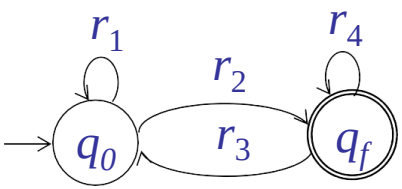
- Remove the state q of the below graph



Connection between Regular Expressions and Regular Languages

- Theorem 3.2: Let L be a regular language. Then there exists a regular expression r such that $L = L(r)$.

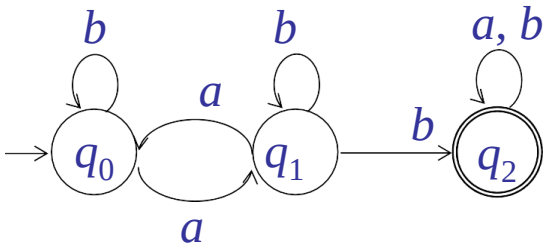
Transition Graph



$r = r_1{}^*r_2(r_4 + r_3r_1{}^*r_2)^*$

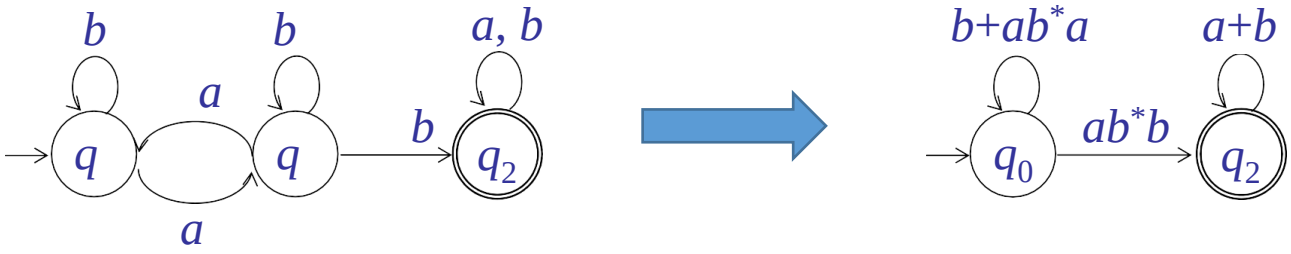
Connection between Regular Expressions and Regular Languages

- Find the regular expression for the below nfa:



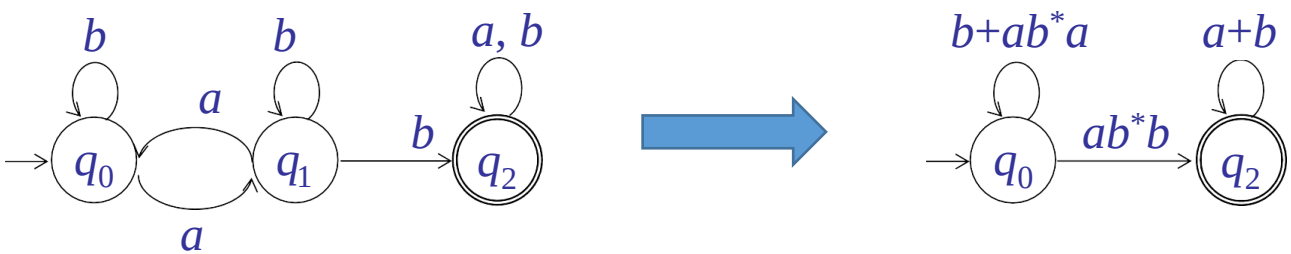
Connection between Regular Expressions and Regular Languages

- Find the regular expression for the below nfa:



Connection between Regular Expressions and Regular Languages

- Find the regular expression for the below nfa:



$$r = (b + ab^*a)^* \, ab^*b(a + b)^*$$

Connection between Regular Expressions and Regular Languages

Regular Expressions for Describing Simple Patterns

- Identifiers, or integers and real numbers in many programming languages.
  - Regular expression sdd^*, where s stands for the sign, with possible values from $\{+ , -, \lambda\}$, and d stands for the digits 0 to 9.
- Used in pattern-matching editor: The vi editor in the UNIX operating system recognizes the command */aba*c/* as an instruction to search the file for the first occurrence of the string *ab*, followed by an arbitrary number of *a*'s, followed by a *c*.

Connection between Regular Expressions and Regular Languages

Exercise

1. Use the construction in Theorem 3.1 to find an nfa that accepts the language L ($ab^*aa + bba^*ab$).
2. Give an nfa that accepts the language L(($a + b$)*b($a + bb$)*).
3. Find **n**fa's that accept the following languages.

   (a) L ($aa^* + aba^*b^*$).

   (b) L (ab ($a + ba$)* ($a + aa$)).

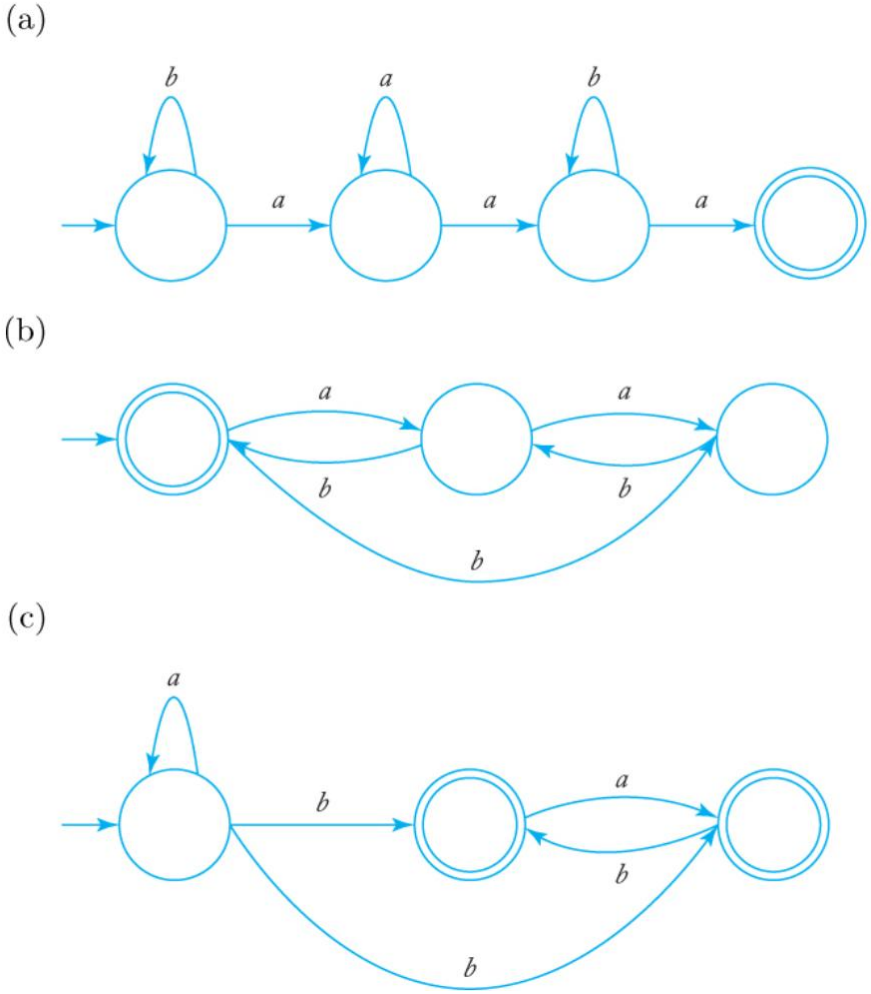
   (c) L (($abab$)* + ($aaa^* + b$)*).

   (d) L (((aa^*)* b)*).

Connection between Regular Expressions and Regular Languages

Exercise

4. Find regular expressions for the languages accepted by the following automata.



Regular Grammars

- Right- and Left-Linear Grammars
- Right-Linear Grammars Generate Regular Languages
- Right-Linear Grammars for Regular Languages
- Equivalence of Regular Languages and Regular Grammars

Regular Grammars

Right- and Left-Linear Grammars

Definition 3.3: A grammar $G = (V, T, S, P)$ is said to be *right-linear* if **all** productions are of the form

$$A \rightarrow x\boldsymbol{B},$$

or

$$A \rightarrow x,$$

where $A, B \in V$, and $x \in T^*$.

A grammar is said to be *left-linear* if **all** productions are of the form

$$A \rightarrow \boldsymbol{B}x,$$

or

$$A \rightarrow x.$$

A regular grammar is one that is either right-linear or left-linear.

Regular Grammars

Example

- The grammar $G_1 = (\{S\}, \{a,b\}, S, P_1)$, with P_1 given as

$$S \rightarrow ab\boldsymbol{S} \mid a$$

is right-linear.

- The grammar $G_2 = (\{S, S_1, S_2\}, \{a, b\}, S, P_2)$, with productions

$$S \rightarrow \boldsymbol{S_1}ab,$$

$$S_1 \rightarrow \boldsymbol{S_1}ab \mid S_2,$$

$$S_2 \rightarrow a,$$

is left-linear.

- Both G_1 and G_2 are regular grammars.

Regular Grammars

- The sequence $S \Rightarrow abS \Rightarrow ababS \Rightarrow ababa$ is a derivation with G_1.
- From this single instance it is easy to conjecture that $L(G_1)$ is the language denoted by the regular expression $r = (ab)^*a$.
- In a similar way, we can see that $L(G_2)$ is the regular language $L(aab(ab)^*)$.

$$S \rightarrow \boldsymbol{S_1}ab,$$
$$S_1 \rightarrow \boldsymbol{S_1}ab \mid S_2,$$
$$S_2 \rightarrow a,$$

Regular Grammars

Linear Grammar

- The grammar $G = (\{S, A, B\}, \{a, b\}, S, P)$ with productions

$$S \to A,$$
$$A \to a\boldsymbol{B} \mid \lambda,$$
$$B \to \boldsymbol{A}b,$$

is **not** regular. Although every production is either in right-linear or left-linear form, the grammar itself is neither right-linear nor left-linear, and therefore is not regular.

A linear grammar is a grammar in which at most one variable can occur on the right side of any production, without restriction on the position of this variable.

Regular Grammars

Right-Linear Grammars Generate Regular Languages

Theorem 3.3: Let $G = (V, T, S, P)$ be a right-linear grammar. Then, $L(G)$ is a regular language.

- *Procedure G_R to nfa()*
  - **Input**: Right-linear grammar $G_R = (V, T, S, P)$
  - **Output**: nfa $M = (Q, \Sigma, \delta, q_0, F)$

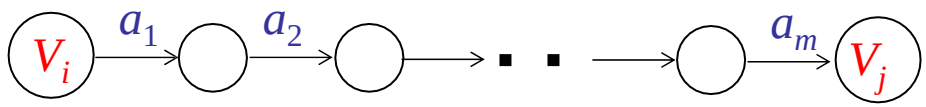
S1. Each variable $V_i \in V$ in G_R is a state labelled as $V_i \in Q$ in nfa M. So, $Q \supset V$.

S2. Initial state of nfa M is the initial variable $S = V_0$.

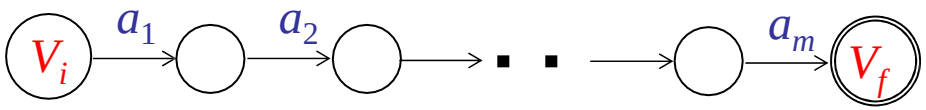
Regular Grammars

S3. If there is a production $V_i \rightarrow a_1a_2\ldots a_m$, the corresponding transition of the automaton will be $\delta^*(V_i, a_1a_2\ldots a_m) = V_f$, where V_f is a final state to be added into the nfa M.

S4. For each production of G_R, $V_i \rightarrow a_1a_2\ldots a_mV_j$, add into the nfa M a state transition $\delta^*(V_i, a_1a_2\ldots a_m) = V_j$.



Represents
$V_i \rightarrow a_1a_2 \ldots a_mV_j$



Represents
$V_i \rightarrow a_1a_2 \ldots a_m$

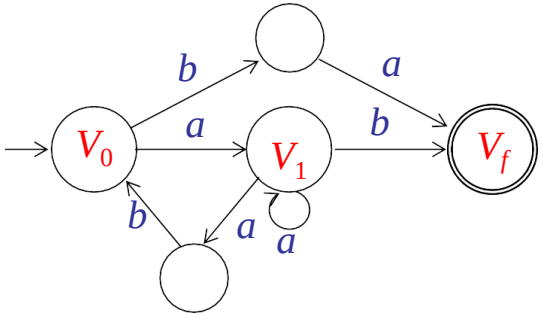
Regular Grammars

Example

- Build up a nfa accepting the language of the below grammar:

$$V_0 \rightarrow aV_1 \mid ba$$
$$V_1 \rightarrow aV_1 \mid abV_0 \mid b$$

- Result



Exercise

- Build up a nfa accepting the language of the below grammar:

$$S \rightarrow aB \mid bA \mid a$$

$$A \rightarrow aaB \mid abS \mid b$$

$$B \rightarrow bbA \mid \lambda$$

Regular Grammars

Right-Linear Grammars for Regular Languages

Theorem 3.4: If L is a regular language on the alphabet Σ, then there exists a right-linear grammar $G = (V,\Sigma,S,P)$ such that $L = L(G)$.

- *Procedure: nfa2G_R*
  - **Input**: nfa $M = (Q, \Sigma, \delta, q_0, F)$
  - **Output**: Right-Linear Grammar $G_P = (V, \Sigma, S, P)$
  - Assume that $Q = \{q_0, q_1, \ldots, q_n\}$ and $\Sigma = \{a_1, a_2, \ldots, a_m\}$.

Regular Grammars

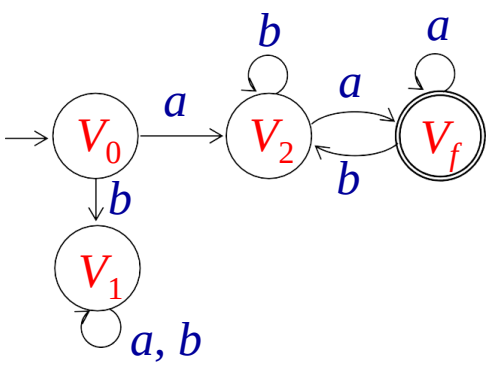
S1. Each state of nfa is a variable in G_R: $V = Q$, $S = q_0$.
S2. For each transition $\delta(q_i, a_j) = q_k$ of M, build an associated Right-Linear production.

$$q_i \rightarrow a_j q_k$$

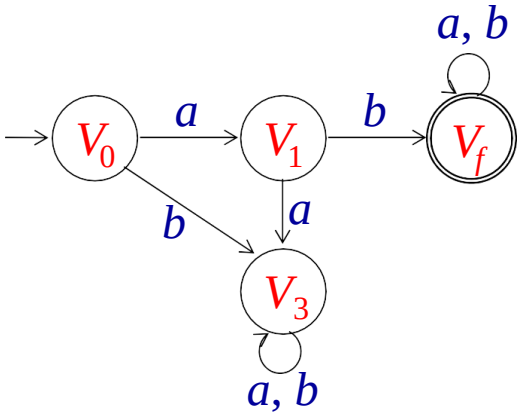
S3. For each state $q_f \in F$, build a production $q_f \rightarrow \lambda$.

Exercise

- Build up a regular grammar of the below nfa:



a)



b)

Regular Grammars

Equivalence of Regular Languages and Regular Grammars

Right-Linear Grammar is equivalent to regular language.

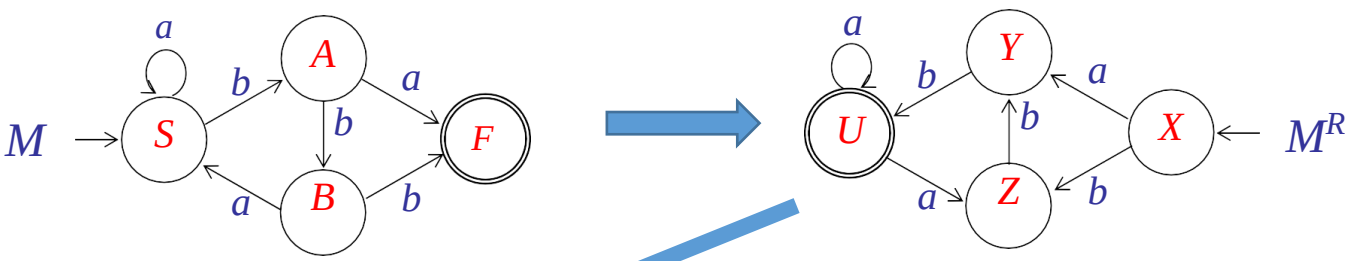
Theorem 3.5: A language L is regular if and only if there exists a left-linear grammar G such that $L = L(G)$.

Theorem 3.6: A language L is regular if and only if there exists a regular grammar G such that $L = L(G)$.

Regular Grammars

- Find a nfa M, the left linear grammar G_L *equivalent to the right linear grammar* G_R:

$$S \to aS \mid bA$$
$$A \to bB \mid a$$
$$B \to aS \mid b$$



G_P^R
$$X \to aY \mid bZ$$
$$Y \to bU$$
$$Z \to bY$$
$$U \to aU \mid aZ \mid \lambda$$

$G^L{}_P$
$$X \to Ya \mid Zb$$
$$Y \to Ub$$
$$Z \to Yb$$
$$U \to Ua \mid Za \mid \lambda$$

Exercise

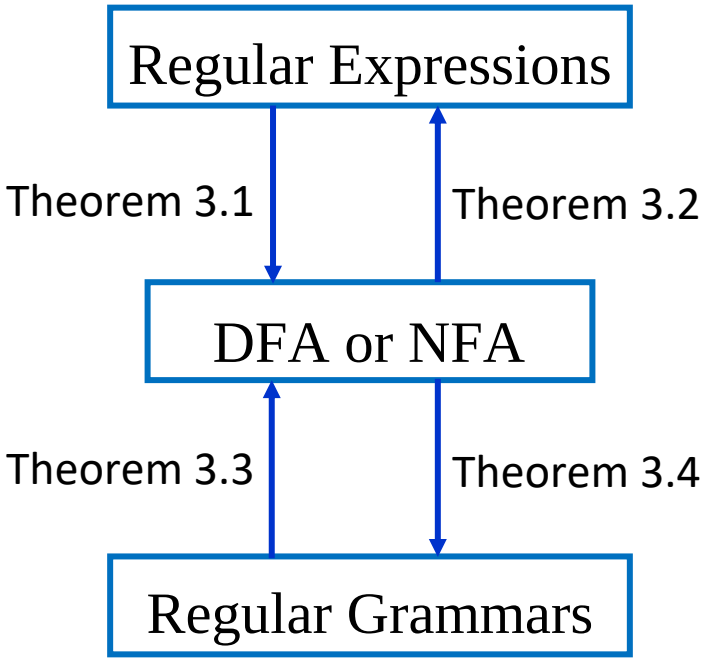
- Find a nfa M and the left linear grammar G_L which are *equivalent to the right linear grammar* G_R:

$$S \rightarrow aB \mid bA \mid a$$

$$A \rightarrow aaB \mid abS \mid b$$

$$B \rightarrow bbA \mid \lambda$$

Regular Grammars



References

1. Peter Linz: An Introduction to Formal Languages and Automata, Fifth edition, Jones & Bartelett learning, 2012.
2. Bài giảng môn Lý thuyết Automat và Ngôn ngữ Hình Thức, Hồ Văn Quân, Đại học Kỹ Thuật TPHCM.